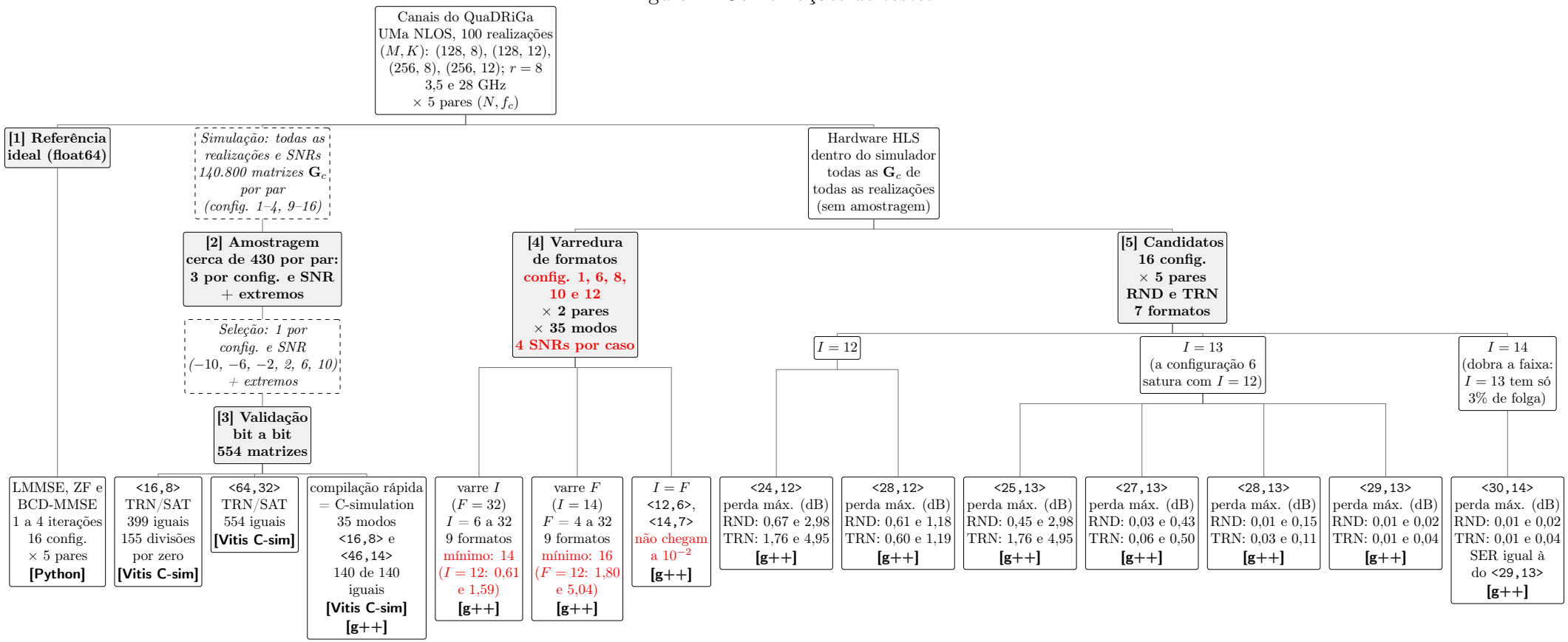
Figura 1: Combinações de testes.



Fonte: a autora. Caixas tracejadas: passos de processamento; cinza: conjuntos de testes; brancas na última linha: cada teste feito. Entre colchetes, a ferramenta: [Python], o simulador em float64; [Vitis C-sim], a C-simulation do Vitis HLS (`vitis-run -csim`); [g++], a compilação rápida. Nenhum teste de SER usou síntese nem co-simulação do RTL; a síntese só estimou o custo de cada formato. `<W,I>` é o formato `ap_fixed<W,I>`: W bits no total, I inteiros (com o sinal) e $F = W - I$ fracionários. RND e TRN são a quantização (arredondar ou truncar) e SAT o estouro (saturar); as configurações e os pares (N, f_c) estão nas tabelas da página seguinte. Perdas de SNR do BCD-MMSE em SER $= 10^{-2}$, em relação à referência ideal, no pior caso das configurações e pares; nos candidatos, "RND: a e b" é a perda com 1 e com 4 iterações; na [4], os mínimos valem para os 35 modos e 1 a 4 iterações, e entre parênteses está a perda com TRN/SAT, com 1 e com 4 iterações. Compilação rápida: o mesmo código C++ do hardware, com os cabeçalhos `ap_fixed` do Vitis, compilado com `g++ -O2`. Só a inversão de $\mathbf{G}_c$ passa pelo hardware; $\hat{\mathbf{R}}_{cc}$ e a matriz $K \times K$ são invertidas em float64.

Definições usadas na árvore

Os conjuntos de testes [1] a [5]

Todos partem dos mesmos canais e das mesmas amostras de ruído (semente fixa), então os resultados de conjuntos diferentes podem ser comparados entre si. Em todos, a matriz invertida pelo hardware é $\mathbf{G}_c = E_s \mathbf{H}_c \mathbf{H}_c^H + \hat{\mathbf{R}}_{cc}$, a do BCD-MMSE em cada DU.

[1] Referência ideal. As curvas de SER de todos os equalizadores (LMMSE, ZF e BCD-MMSE com 1 a 4 iterações) com a inversão feita em ponto flutuante de 64 bits, sem nenhum erro de hardware. É o melhor resultado possível e a base de comparação dos outros conjuntos. Feito para as 16 configurações e os 5 pares.

[2] Amostra das matrizes. O simulador rodou todas as realizações e SNRs das configurações 1 a 4 e 9 a 16, o que dá 140.800 matrizes $\mathbf{G}_c$ por par (105.600 de 16×16 e 35.200 de 32×32), e calculou as métricas de todas. Dessas, ficaram cerca de 430 por par, gravadas em float64: 3 de cada configuração e SNR, sorteadas, mais as mais difíceis para o hardware (maior valor, pior condicionamento, menor autovalor e maior valor da inversa). Não é um teste: é o material de entrada do conjunto [3]. Os conjuntos [4] e [5] não usam essa amostra: neles, o hardware inverte todas as matrizes de todas as realizações.

[3] Validação bit a bit. Responde se o hardware faz exatamente a conta que o modelo do simulador faz. Da amostra [2], uma nova seleção ficou com 554 matrizes: em cada par, tamanho e configuração, uma matriz de cobertura em cada SNR de -10, -6, -2, 2, 6 e 10 dB, mais todos os extremos. Cada uma foi invertida de três formas, no mesmo formato: pela C-simulation do Vitis HLS (o hardware), pelo modelo Python do hardware (`golden_cholesky.py`) e pelo caminho que o simulador usa. Os bits dos três foram comparados. Em `<16,8>` e `<64,32>`, nenhuma matriz deu bits diferentes. Em `<16,8>`, 155 matrizes têm algum valor acima de 128, a faixa desse formato: o valor satura, um pivô da decomposição fica menor ou igual a zero e a conta divide por zero. Isso acontece igual nos três ("divisão por zero nos três"), mas nesses casos não há bits para comparar; por isso elas são contadas à parte das 399 iguais. Também conferiu que a compilação rápida, usada em [4] e [5], dá os mesmos bits que a C-simulation (ver "Compilação rápida e C-simulation", abaixo).

[4] Varredura de formatos. É uma **exploração**, não uma validação: procura em que região estão os formatos e sugere os candidatos da [5]. O próprio hardware HLS fez a inversão dentro do simulador: o simulador manda cada $\mathbf{G}_c$ ao hardware e usa a inversa que ele devolve. Variou o número de bits inteiros (com 32 fracionários), o de fracionários (com 14 inteiros) e os 35 modos, nos casos extremos escolhidos pela métrica do ponto de operação (ver "Como os casos extremos da [4] foram escolhidos"): configurações 1, 6, 8, 10 e 12, nos pares (192; 3,5 GHz) e (168; 28 GHz), só nas SNRs em volta do cruzamento com SER $= 10^{-2}$. O menor formato da grade que fica abaixo de 0,1 dB tem $I = 14$ e $F = 16$, nos 35 modos e com 1 a 4 iterações. Com $I = 12$, as configurações 6 e 8 saturam; $I = 13$, $F = 14$ e $F = 15$ não estão na grade.

[5] Candidatos. É a **validação** dos formatos: confere os valores que ficaram entre os pontos da grade da [4] ($I = 12$ e 13; $F = 12$, 14, 15 e 16) em todas as 16 configurações e os 5 pares, também com o hardware fazendo a inversão. Com 12 bits inteiros, a configuração 6 satura em -10 dB; com 13, não. Com 13 inteiros, foram testados 12, 14, 15 e 16 fracionários: `<27,13>` (14 fracionários) perde no máximo 0,06 dB com 1 iteração e até 0,5 dB com 4; `<28,13>` (15 fracionários) perde até 0,07 dB com 3 iterações e 0,11 dB com 4; `<29,13>` (16 fracionários) fica a no máximo 0,04 dB da referência ideal [1] em todos os casos, com 1 a 4 iterações. Por fim, `<30,14>` (um bit inteiro a mais, para dar folga à faixa) deu exatamente a mesma SER do `<29,13>`.

Diferença entre [4] e [5]. As duas usam o mesmo método (o hardware inverte todas as matrizes dentro do simulador, e se mede a perda em relação a [1]), mas respondem perguntas diferentes. A [4] é larga nos formatos e estreita nos casos: testa muitos formatos e todos os modos, só nos casos extremos, para achar onde está o limite de cada eixo. A [5] é estreita nos formatos e larga nos casos: pega os poucos formatos que a [4] apontou e confere em todas as configurações e pares.

	[4] Varredura de formatos	[5] Candidatos
Papel	exploração	validação
Pergunta	quantos bits e qual modo?	os escolhidos funcionam em todos os casos?
Formatos	19 (um eixo de cada vez)	7 (os dois eixos no limite)
Modos	os 35	AP_RND e AP_TRN, com AP_SAT
Configurações	5 (1, 6, 8, 10 e 12)	as 16
Pares (N, f_c)	2	os 5
SNRs	4 por caso	as 11 da grade
Rodadas	6.650	1.120 (320 + 320 + 160 + 320)
Tempo	cerca de 7 h	cerca de 4 h

A [4] sozinha deixaria duas dúvidas: se os dois mínimos juntos ainda funcionam, e se as configurações que ficaram de fora têm um caso pior. A [5] respondeu às duas: os valores entre os pontos da grade funcionam juntos, e o pior caso nas 16 configurações continuou sendo o das configurações 6 e 8, que já estavam na [4]. Mesmo assim, a [4] cobre só 5 das 16 configurações e 2 dos 5 pares, e o formato final vem só da [5].

O que vem de cada uma.

Conclusão	De onde vem	Vale nas 16 configurações?
$I = 14$ e F de 14 a 16, conforme as iterações	[5]	sim
truncar (AP_TRN) em vez de arredondar (AP_RND)	[5]	sim (os dois rodaram)
estouro AP_SAT	[5]	sim (só o AP_SAT rodou)
com $I \geq 14$, o modo de estouro não muda nenhum bit da SER	[4]	não: nas 1, 6, 8, 10 e 12
com $I = 12$, o modo de estouro muda a perda, mas nenhum fica abaixo de 0,1 dB	[4]	não: nas 1, 6, 8, 10 e 12
as 5 variantes de arredondamento perdem quase o mesmo (até 0,004 dB de diferença)	[4]	não: nas 1, 6, 8, 10 e 12
AP_TRN_ZERO perde mais que AP_TRN com poucos bits fracionários	[4]	não: nas 1, 6, 8, 10 e 12
arredondar não diminui o menor F da grade (16 nos dois)	[4] e [5]	sim, pela [5]

O formato final, `<30,14>` com AP_TRN e AP_SAT, foi todo testado na [5] e não depende das três últimas linhas. Sobre o estouro há um argumento a mais: ele só age quando um valor passa da faixa. Com $I = 14$, a SER foi idêntica à de $I = 13$, o que indica que nada saturou nos 16 casos; nesses casos, trocar AP_SAT por AP_WRAP não mudaria nenhum bit. Na [4], isso foi testado nos 5 casos: com $I = 14$, os 5 modos de estouro deram exatamente a mesma SER, inclusive nas configurações 6 e 8, os piores casos. Nas outras 11 configurações, continua uma dedução. Com $I = 12$, o modo de estouro muda o resultado (com `<44,12>` e 1 iteração, de 0,17 a 0,64 dB, conforme o modo), porque aí há saturação. Com `<26,14>` ($F = 12$), AP_TRN_ZERO perde 2,50 dB com 1 iteração, contra 1,80 do AP_TRN; com `<30,14>` e 4 iterações, 0,054 contra 0,04 dB.

Compilação rápida e C-simulation

Nos testes, "o hardware" é o código C++ do HLS (a decomposição de Cholesky em `hardware/cholesky_hardware/`), executado no computador. Ele foi compilado de duas formas, sempre com os mesmos fontes e os mesmos cabeçalhos `ap_fixed` do Vitis:

- **C-simulation do Vitis** (`vitis-run -csim`): o compilador do Vitis, com as bibliotecas de simulação do HLS. É a forma padrão de verificar um algoritmo HLS e a referência da validação [3]. Leva de 20 a 290 ms por inversão;

- **compilação rápida** (`g++ -O2`): o compilador comum do Linux, com otimização. Leva de 0,6 a 21 ms por inversão, de 15 a 35 vezes menos.

Os conjuntos [4] e [5] invertem todas as matrizes de todas as rodadas, mais de 30 milhões de inversõesno hardware (21 milhões na [4] e 12 milhões na [5]): com a C-simulation, levariam mais de 100 horas; com a compilação rápida, ~~uma noite~~ cerca de 11 horas (7 na [4] e 4 na [5]). Antes de usá-la, a folha "compilação rápida = C-simulation" da árvore conferiu que as duas dão os mesmos bits: as mesmas 30 matrizes em cada um dos 35 modos, nos formatos `<16,8>` e `<46,14>`, em 16×16 e 32×32, deram bits iguais nos 140 casos.

Nos dois casos, o que roda é o código C++, não o circuito sintetizado. ~~A síntese e a co-simulação do RTL, que mostrariam o comportamento do circuito gerado pelo HLS, ainda não foram feitas.~~ A co-simulação do RTL, que mostraria o comportamento do circuito gerado pelo HLS, ainda não foi feita. A síntese foi feita depois, só para estimar o custo de cada formato (ver "Síntese e fronteira de Pareto").

Como o hardware entra no simulador

Nos conjuntos [4] e [5], o hardware fica aberto como um processo separado durante toda a simulação. A cada matriz $\mathbf{G}_c$, o simulador manda os valores em float64 ao hardware; o hardware converte para ponto fixo (com a quantização e o estouro do formato), faz a decomposição de Cholesky e devolve a inversa; o simulador continua a equalização com essa inversa. Antes de usar esse caminho, ele foi conferido contra a validação [3] (mesmos bits nas 554 matrizes) e contra o modelo Python (mesma SER).

Quando o hardware divide por zero, ele avisa o simulador e é reiniciado, e a inversão daquela matriz vira "não é um número" (NaN). O NaN passa para as DUs seguintes do BCD-MMSE, e a SER daquele ponto fica perto de 1. Um hardware real devolveria valores errados, e não NaN, então nesses pontos a SER simulada é só uma indicação de falha.

Só a inversão de $\mathbf{G}_c$ (o módulo 2) passa pelo hardware. A inversão de $\hat{\mathbf{R}}_{cc}$ (módulo 1) e a da matriz $K \times K$ (módulo 3) continuam em float64.

Formato de ponto fixo

O hardware representa cada número em ponto fixo, no tipo `ap_fixed<W,I>` do Vitis HLS:

- W é o número total de bits;
- I é o número de **bits inteiros**, com o bit de sinal; define a faixa de valores, de -2^{I-1} a 2^{I-1} (com $I = 13$, de -4096 a 4096);
- $F = W - I$ é o número de **bits fracionários**; define a precisão, o menor passo entre dois valores, 2^{-F} (com $F = 16$, cerca de $1{,}5 \times 10^{-5}$).

Por exemplo, `<29,13>` é `ap_fixed<29,13>`: 29 bits, 13 inteiros e 16 fracionários. O formato atual do hardware é `<16,8>`.

Uma régua ajuda a imaginar. Os bits inteiros dizem **até onde a régua vai**: um número maior que a régua estoura. Os bits fracionários dizem **quão finas são as marcações**: com marcações grossas, cada medida sai arredondada.

Quando um valor não cabe no formato, dois modos decidem o que acontece:

- **quantização**, para os bits que sobram à direita: TRN trunca (AP_TRN, o modo atual do hardware) e RND arredonda (AP_RND). Há 7 modos ao todo;
- **estouro**, para valores além da faixa: SAT satura no maior valor (AP_SAT, o modo atual) e WRAP dá a volta. Há 5 modos ao todo.

A varredura [4] testou as $7 \times 5 = 35$ combinações:

- quantização: AP_TRN e AP_TRN_ZERO (truncam); AP_RND, AP_RND_ZERO, AP_RND_MIN_INF, AP_RND_INF e AP_RND_CONV (arredondam, com regras diferentes para o empate);

- estouro: AP_SAT, AP_SAT_ZERO e AP_SAT_SYM (saturam); AP_WRAP e AP_WRAP_SM (dão a volta).

Ponto fixo de 64 bits e float64 usam os mesmos 64 bits por parte, mas organizados de jeitos diferentes: o ponto fixo tem um passo fixo (2^{-F}), e o float64 tem 1 bit de sinal, 11 de expoente e 52 de mantissa, com um passo proporcional ao valor. O mesmo número fica com bits diferentes nos dois. Por isso o hardware é comparado bit a bit só com o modelo no mesmo formato ([3]); com a referência em float64 ([1]), a comparação é pelo valor, ou seja, pelo erro e pela SER.

Critério de avaliação

A medida usada em [4] e [5] é a **perda de SNR**: quantos dB a mais de SNR o BCD-MMSE com o hardware precisa para chegar à mesma SER que a referência ideal [1], na SER $= 10^{-2}$ (a SNR em que cada curva cruza 10^{-2} é interpolada entre os pontos simulados). Um formato é considerado igual à referência quando a perda fica abaixo de 0,1 dB no pior caso. O valor de 10^{-2} e o limiar de 0,1 dB são escolhas deste trabalho.

O que é a grade. "Grade" é a lista de valores em que uma variável foi testada. A **grade de SNR** tem 11 pontos, de -10 a 10 dB, de 2 em 2 dB: a SER só é simulada nesses pontos. Entre dois pontos, a curva não é conhecida; por isso a SNR do cruzamento com 10^{-2} é interpolada, e uma curva que cruza abaixo de -10 dB fica fora da grade. A **grade de formatos** da [4] é a lista de valores de I e F testados; a resposta da [4] só pode ser um desses valores.

A grade de SNR da [4]. Para gastar menos tempo, a [4] rodou só os pontos da grade em volta do cruzamento com 10^{-2}, mais dois pontos à direita, onde a curva com poucos bits cruzaria. São 4 SNRs por caso: de -10 a -4 dB nas configurações 6 e 8, de -6 a 0 dB nas 10 e 12 e de -4 a 2 dB na 1. O simulador sorteia o ruído de cada SNR em sequência, então pular SNRs muda os sorteios. Por isso a referência ideal foi simulada de novo junto, nas mesmas SNRs e com a mesma semente, e a perda da [4] é medida contra essa referência.

Ponto de operação. É a SNR em que a curva da referência ideal [1] cruza SER $= 10^{-2}$: o sinal mínimo para errar 1 símbolo a cada 100. Cada configuração, par e número de iterações tem o seu. Com QPSK, ele fica mais baixo (perto de -10 dB) do que com 16-QAM. É nesse ponto que a perda é medida, então são as matrizes dessa SNR que pesam no resultado.

A medida é relativa porque a norma não define um valor de SER aceitável. A TS 38.104 (Sec. 8.2) define o desempenho do receptor em vazão, depois do código LDPC e com retransmissões (HARQ): pelo menos 70% da vazão máxima de um canal de referência, em uma SNR dada. A SER simulada aqui é antes da decodificação, e o quanto de erro o código corrige depende da taxa do código. Também não há um requisito para a saída do equalizador: o EVM da norma (Sec. 6.5.2: 17,5% para QPSK, 12,5% para 16-QAM) mede a qualidade do transmissor da estação base. Por isso o critério é que o hardware não mova a curva: se a SER é a mesma da referência, o resto da cadeia recebe o mesmo sinal que receberia com a inversão ideal.

Como a varredura [4] foi montada

A [4] mede um efeito de cada vez. Cada eixo é variado com o outro em folga, para que qualquer perda venha só do eixo que está mudando.

- **Varredura de I, com $F = 32$ fixo.** Com 32 bits fracionários, a precisão sobra: com $F = 32$, o erro contra a inversa em float64 fica perto de 10^{-8}. Se a curva piora quando I diminui, a causa é o estouro (a saturação). O valor 32 também é o maior que o modelo reproduz bit a bit (o `hls::sqrt` só usa o algoritmo modelado com até 32 bits fracionários).

- **Varredura de F, com $I = 14$ fixo.** Os maiores valores das matrizes chegam a cerca de 3.960. Para não saturar, é preciso a faixa de ± 4096 ($I = 13$); com $I = 14$, a faixa vai a ± 8192, com folga. Se a curva piora quando F diminui, a causa é a falta de precisão (a quantização).

Na régua: na varredura de I, as marcações são finíssimas e só o comprimento muda; na de F, a régua é bem comprida e só as marcações mudam.

Testar todas as combinações de I e F daria $9 \times 9 = 81$ formatos $\times$ 35 modos, mais de quatro vezes o que foi feito. O método de um eixo de cada vez não mostra o que acontece com os dois eixos no limite ao mesmo tempo; isso ficou para a [5].

Como os candidatos [5] foram escolhidos

Na [5], a ideia é outra: achar o **menor** formato que funciona, porque cada bit a mais custa área e energia no hardware (multiplicadores e somadores crescem com a largura dos números). Os valores fixos da [4] eram só folgas do teste, e não candidatos.

Pelo critério de 0,1 dB, a [4] deu, no pior caso das configurações 1, 6, 8, 10 e 12:

Modo	menor I (com $F = 32$)	menor F (com $I = 14$)
os 35 modos, com 1 a 4 iterações	14	16

Como ler essa tabela. "Menor I" é o menor número de bits inteiros da grade em que o hardware ainda fica igual à referência ideal [1], com $F = 32$. "Menor F" é o menor número de bits fracionários em que isso acontece, com $I = 14$. "Igual à referência" quer dizer perda de SNR abaixo de 0,1 dB em SER $= 10^{-2}$. Alguns detalhes:

- O valor é o **pior caso**: o mínimo é calculado em cada configuração e em cada par, e a tabela mostra o maior deles. Cada caso pede um tamanho diferente (tabela abaixo); o pior vem das configurações 6 e 8.
- A resposta só pode ser um **ponto da grade**: $I = 6, 8, 10, 12, 14, 16, 20, 24$ e 32; $F = 4, 6, 8, 10, 12, 16, 20, 24$ e 32. Não havia $I = 13$, $F = 14$ nem $F = 15$. Então "menor $F = 16$" quer dizer que 12 não passou e 16 passou, e o mínimo de verdade pode estar entre os dois. Por isso a [5] testou $I = 13$ e $F = 14$ e 15.
- Cada mínimo vale com o **outro eixo em folga**. Com $F = 32$, a precisão sobra, e a perda ao diminuir I vem da saturação. Com $I = 14$, a faixa sobra, e a perda ao diminuir F vem da falta de precisão. A tabela não garante nada com os dois eixos no limite ao mesmo tempo; isso também ficou para a [5].

Por caso. Com AP_TRN e AP_SAT; os dois pares deram o mesmo resultado em todos os casos.

Configuração	O que é	menor I	menor F, TRN	menor F, RND
1	16-QAM, $M_c = 16$, IoT de 10 dB	8	12	10 (1 iter.); 12 (4 iter.)
10 e 12	16-QAM, $M_c = 32$, IoT de 20 dB	12	16	12 (1 iter.); 16 (4 iter.)
6 e 8	QPSK, $M_c = 16$, IoT de 20 dB	14	16	16

A configuração 1 trabalha numa SNR mais alta (perto de $-2{,}5$ dB), onde $\mathbf{G}_c$ tem valores menores, e 8 bits inteiros bastam. As 6 e 8 trabalham perto de -9 dB, onde $\mathbf{G}_c$ é maior, e saturam com $I = 12$. A varredura de I deu o mesmo com TRN e RND e com 1 a 4 iterações.

Com $I = 12$, a perda vai de 0,61 dB (1 iteração) a 1,59 dB (4 iterações), nas configurações 6 e 8. Com $F = 12$, truncar perde 1,80 dB e arredondar 0,22 dB com 1 iteração: arredondar ajuda, mas nenhum dos dois passa, e os dois só passam com $F = 16$. Então, nos casos extremos, arredondar não diminuiu o menor F da grade.

Daí os candidatos:

- estouro AP_SAT, o modo atual do hardware, já que, com $I \geq 14$, o modo de estouro não mudou nenhum bit na [4];
- quantização AP_TRN (o modo atual do hardware, e o mais barato) e AP_RND (o arredondamento mais simples; precisa de um somador a mais), um representante de cada família;
- $I = 12$, para ver nas 16 configurações onde ele satura: `<24,12>` ($F = 12$) e `<28,12>` ($F = 16$);
- $I = 13$, entre o 12 que satura e o 14 que passa, com $F = 12$, 14, 15 e 16.

Os três valores de I testados:

I	Faixa	Papel
12	±2048	satura nas configurações 6 e 8, como na [4]
13	±4096	o mínimo que não satura em nenhum dos 16 casos; folga de só 3%
14	±8192	o menor da grade da [4]; formato final, com o dobro da faixa

Com $I = 13$, foram testados 12, 14, 15 e 16 bits fracionários (`<25,13>`, `<27,13>`, `<28,13>` e `<29,13>`). O maior valor das matrizes, 3.960 (configurações com IoT de 20 dB, em −10 dB), fica a só 3% do limite de 4.096. Uma SNR um pouco abaixo da grade simulada já estouraria $I = 13$. Por isso foi testado também `<30,14>`.

I e F agem separados. Com o mesmo F, mudar I não mudou a perda: `<24,12>` e `<25,13>` perdem o mesmo, e `<30,14>` deu exatamente a mesma SER do `<29,13>` nas 160 rodadas. Ou seja, quando não há saturação, o bit inteiro a mais não mexe na precisão. Por isso I pode ser escolhido pela faixa e F pela precisão, um de cada vez.

Formato	I	F	AP_TRN		AP_RND		O que limita
			1 iter.	4 iter.	1 iter.	4 iter.	
`<24,12>`	12	12	1,76	4,95	0,67	2,98	F
`<28,12>`	12	16	0,60	1,19	0,61	1,18	I: satura nas configurações 6 e 8
`<25,13>`	13	12	1,76	4,95	0,45	2,98	F
`<27,13>`	13	14	0,06	0,50	0,03	0,43	F, a partir de 2 iterações
`<28,13>`	13	15	0,03	0,11	0,01	0,15	F, só com 4 iterações
`<29,13>`	13	16	0,01	0,04	0,01	0,02	nada
`<30,14>`	14	16	0,01	0,04	0,01	0,02	nada (SER igual à do `<29,13>`)

Perda máxima de SNR (dB) em SER $= 10^{-2}$, no pior caso das 16 configurações e 5 pares, em relação à referência ideal [1]. Todos com AP_SAT.

Resultado e o que ainda não está provado

- **Bits inteiros**: $I = 14$. $I = 13$ é o mínimo que não satura, mas só cobre a grade simulada (até −10 dB), com 3% de folga; $I = 14$ dobra a faixa sem mudar a SER.
- **Bits fracionários**, conforme o número de iterações do BCD-MMSE: 14 bastam com 1 iteração (como no artigo base; 0,06 dB), 15 com até 3 (0,07 dB) e 16 com 4 (0,04 dB). Com mais iterações, o erro do ponto fixo se acumula: o `<27,13>` chega a 0,5 dB com 4 iterações.
- **Formato final**: `<30,14>` com AP_TRN e AP_SAT, para 1 a 4 iterações, com perda máxima de 0,04 dB nas 16 configurações e 5 pares. Se o número de iterações for menor, `<29,14>` (até 3) ou `<28,14>` (1 iteração).
- Arredondar não economizou bits de forma consistente: ajuda com $F = 12$, mas com $F = 15$ e 4 iterações fica pior que truncar (0,15 contra 0,11 dB). Então o truncamento, que é o modo atual e o mais barato, pode ser mantido.
- **Custo**: pela síntese sem diretivas, `<28,13>` usa 112 DSP e `<30,14>` usa 184. Se o BCD-MMSE usar até 3 iterações, `<28,13>` é o mais barato que atende o critério, mas com a folga de só 3% na faixa. Com 4 iterações, é preciso subir para 184 DSP (ver "Síntese e fronteira de Pareto").

O que ainda não está provado: `<29,14>` e `<28,14>` não foram rodados (o resultado deles vem de `<28,13>` e `<27,13>`, pela separação entre I e F); a grade de SNR para em −10 dB, e nas configurações 5, 6 e 7 a curva cruza 10^{-2} perto dessa borda; se $F = 13$ bastaria com 1 iteração (com $F = 12$ já perde 0,45 dB); ~~se os outros modos~~

~~de quantização e de estouro se comportam como na [4] também nas 16 configurações~~ se os outros modos se comportam, nas outras 11 configurações, como nos 5 casos da [4]; o formato dos módulos 1 e 3, que não passaram pelo hardware; e o custo de cada formato no circuito~~, que depende da síntese no HLS~~: a síntese feita é só a C synthesis do Vitis HLS, sem diretivas; faltam as diretivas, a implementação no Vivado e a co-simulação.

Síntese e fronteira de Pareto

A síntese estima o custo de cada formato no circuito. Foi feita só a C synthesis do Vitis HLS 2026.1 (o HLS gera o RTL e estima ciclos, frequência e recursos), sem implementação no Vivado. O topo da síntese é a função `cholesky_inverse`: só a matriz $\mathbf{G}_c$ entra e só a inversa sai, com as mesmas etapas da decomposição de Cholesky que rodaram nos testes. Ela foi conferida contra o código dos testes nas 554 matrizes da seleção [3], com `<16,8>`: os mesmos bits em todas. Condições da síntese:

- placa XCU50-FSVH2104-2-E (Alveo U50);
- relógio de 10 ns (100 MHz). Essa frequência ainda não tem fonte nem justificativa;
- AP_TRN e AP_SAT, matriz 16×16 ($M_c = 16$), salvo onde indicado;
- sem nenhuma diretiva de HLS. Por padrão, o Vitis aplica sozinho um pipeline nos laços internos; isso foi desligado (`syn.compile.pipeline_loops=0`), para ter a versão de partida antes das otimizações.

O formato `<24,12>`. Primeira síntese, antes da fronteira de Pareto:

	16×16	32×32	16×16, pipeline automático
Ciclos por inversão	65.500	382.732	52.428
Tempo a 100 MHz	655 μs	3,83 ms	524 μs
Frequência máxima estimada	139,8 MHz	139,6 MHz	139,8 MHz
DSP	90	90	106
BRAM	11	22	11
LUT	15.348	15.429	16.187
FF	5.018	5.115	5.530

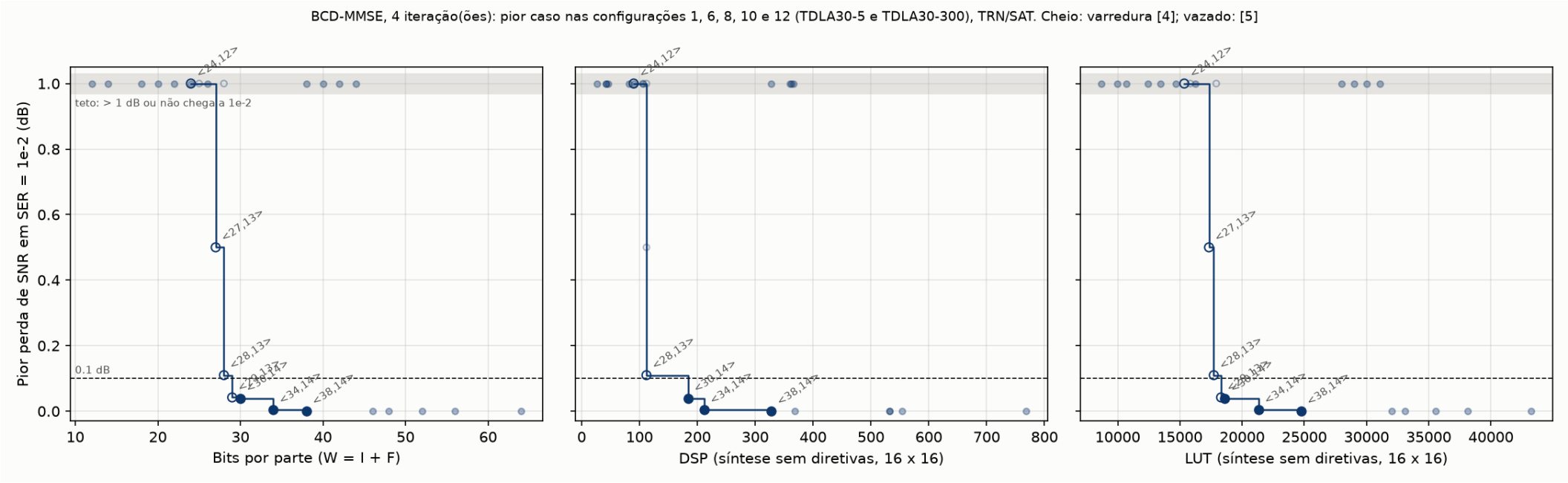
Sem diretivas, o HLS usa o mesmo hardware para todas as contas, uma depois da outra. Por isso dobrar a matriz multiplica os ciclos por cerca de 6, e os recursos quase não mudam. A decomposição LU e a substituição para a frente são de 80 a 85% dos ciclos nos dois tamanhos. A frequência máxima é só uma estimativa da síntese; o valor real sai da implementação.

Comparação com o requisito de tempo. Pela planilha de restrições deste trabalho, cada módulo de inversão precisa de um intervalo entre inversões de no máximo $T_c/275$. A 100 MHz, isso dá 9.090 ciclos na condição mais lenta (TDLA30-5), 152 na TDLA30-300 e 114 na TDLB100-400. A versão sem diretivas fica 7 vezes acima (16×16) e 42 vezes acima (32×32) da condição mais lenta. As diretivas ainda vão ser testadas.

Fronteira de Pareto. Os 25 formatos da [4] e da [5] foram sintetizados nas mesmas condições. Para cada formato, a perda é a pior entre os casos da [4] (configurações 1, 6, 8, 10 e 12; pares (192; 3,5 GHz) e (168; 28 GHz)). Os formatos `<27,13>`, `<28,13>` e `<29,13>` não estão na [4]; a perda deles vem da [5], nos mesmos casos. Um formato está na fronteira quando nenhum outro custa o mesmo ou menos e perde menos.

Formato	DSP	LUT	Ciclos	Pior perda (dB), por iterações			
				1	2	3	4
<24,12>	90	15.348	65.500	1,76	3,20	4,20	4,95
<26,14>	106	16.215	67.589	1,80	3,47	4,15	5,04
<27,13>	112	17.334	77.872	0,06	0,15	0,27	0,50
<28,13>	112	17.721	79.720	0,03	0,05	0,07	0,11
<29,13>	184	18.304	81.600	0,01	0,02	0,02	0,04
<30,14>	184	18.606	82.532	0,01	0,02	0,03	0,04
<34,14>	212	21.327	89.763	0,00	0,00	0,00	0,00
<44,12>	366	31.058	113.931	0,61	0,99	1,38	1,59

Os outros formatos não chegam a SER = 10^{-2}, ou custam mais sem perder menos. O <38,14> (328 DSP) ainda fica na fronteira, mas só ganha 0,004 dB sobre o <34,14>.



Fronteira com 4 iterações. Cheio: formatos da [4]; vazado: da [5]. A linha tracejada é o limiar de 0,1 dB; no teto ficam os formatos que perdem mais de 1 dB ou não chegam a 10^{-2}.

Lendo a fronteira:

- **Os DSP sobem em degraus.** Formatos com o mesmo W usaram os mesmos DSP (<24,12> e <24,14>: 90; <28,12> e <28,13>: 112). De $W = 28$ para $W = 29$, o número salta de 112 para 184 (64% a mais). Os LUT sobem quase em linha reta.
- **Até 3 iterações**: <28,13> (112 DSP) é o mais barato abaixo de 0,1 dB. Ele custa o mesmo que <27,13> e perde menos.

- **Com 4 iterações**: `<28,13>` perde 0,11 dB. É preciso subir para o degrau de 184 DSP, com `<29,13>` ou `<30,14>`. Os dois custam quase o mesmo, e `<30,14>` tem o dobro da faixa.
- **O bit inteiro de folga custa um degrau com 15 bits fracionários.** `<29,14>` teria $W = 29$, então deve cair no degrau de 184 DSP, como o `<29,13>`. Isso é uma dedução: `<29,14>` não foi sintetizado nem simulado.

Ressalvas: o custo é o da versão sem diretivas, e os degraus podem mudar com elas; as perdas de `<27,13>`, `<28,13>` e `<29,13>` vêm de outra rodada (a [5], com a grade inteira de SNR e outros sorteios); e `<28,13>` com 4 iterações fica muito perto do limiar (0,11 dB).

Configurações

Os parâmetros do sistema que mudam entre as simulações. As configurações 1, 2, 3, 5, 9 e 13 são as do artigo base (Zhao et al., Fig. 8); as outras cruzam $K = 12$ e IoT de 20 dB com as outras bases. A numeração segue (M, C); com $M = 128$, primeiro 16-QAM e depois QPSK; dentro de cada grupo, K e depois a IoT.

$$M_c = M/C \quad \text{(antenas por } \textit{cluster}\text{)}, \qquad r = 8 \text{ interferentes em todas.}$$

Configuração	M	C	M_c	K	IoT (dB)	Modulação	Observação
1	128	8	16	8	10	16-QAM	referência do artigo; caso típico da [4]
2	128	8	16	8	20	16-QAM	do artigo
3	128	8	16	12	10	16-QAM	do artigo
4	128	8	16	12	20	16-QAM	
5	128	8	16	8	10	QPSK	do artigo; mesma matriz da 1
6	128	8	16	8	20	QPSK	mesma matriz da 2; pior caso da [4] e dos candidatos, com a 8
7	128	8	16	12	10	QPSK	mesma matriz da 3
8	128	8	16	12	20	QPSK	mesma matriz da 4; caso extremo da [4]
9	256	8	32	8	10	16-QAM	do artigo
10	256	8	32	8	20	16-QAM	caso extremo da [4] ($M_c = 32$)
11	256	8	32	12	10	16-QAM	~~extremo de precisão pelas métricas da matriz~~
12	256	8	32	12	20	16-QAM	caso extremo da [4] ($M_c = 32$)
13	256	16	16	8	10	16-QAM	do artigo
14	256	16	16	8	20	16-QAM	
15	256	16	16	12	10	16-QAM	
16	256	16	16	12	20	16-QAM	

M: antenas da estação base; C: número de *clusters* (DUs); K: usuários atendidos; IoT: interferência sobre o ruído térmico.

Por que nem todas as configurações estão em [2] e em [4]

Amostra [2]: sem as configurações 5 a 8. O hardware inverte $\mathbf{G}_c = E_s \mathbf{H}_c \mathbf{H}_c^H + \hat{\mathbf{R}}_{cc}$, que depende só do canal, do ruído e de E_s: os símbolos transmitidos, e portanto a modulação, não entram nela. As configurações 5 a 8 são iguais às 1 a 4, só que com QPSK em vez de 16-QAM. Com a mesma semente, usam os mesmos canais e as mesmas amostras de ruído, e o hardware recebe as mesmas matrizes. Amostrá-las só repetiria matrizes que já estão na amostra.

Varredura [4]: as configurações 1, 6, 8, 10 e 12. Com as 16 configurações e os 5 pares, os $19 \times 35 = 665$ formatos e modos dariam cerca de 53 mil rodadas, ou semanas de simulação. A varredura ficou com 5 configurações e 2 pares, escolhidos pela métrica do ponto de operação (próxima seção): as configurações 6 e 8 (QPSK, IoT de 20 dB), que ficam no topo da lista; as 10 e 12 (16-QAM, $M_c = 32$), porque a métrica falha com $M_c = 32$; e a 1, como caso típico. Os pares são os de maior perda, (192; 3,5 GHz) e (168; 28 GHz). Com só 4 SNRs por caso, foram 6.650 rodadas em cerca de 7 horas.

Como os casos extremos da [4] foram escolhidos

Uma primeira ideia seria escolher pelas métricas das matrizes em todas as SNRs da grade: maior valor, número de condição, maior valor da inversa. Elas apontam as configurações 11 e 12 ($M_c = 32$), que têm o maior número de condição e os maiores valores ao longo da grade. Mas a perda é medida só no ponto de operação, e a modulação muda onde ele fica. A matriz da configuração 6 é a mesma da 2, mas, com QPSK, a curva chega a SER $= 10^{-2}$ perto de -10 dB, justamente onde as matrizes têm os maiores valores. Com 16-QAM, essa região fica com SER entre 0,5 e 0,9, e não pesa na medida em 10^{-2}. Ou seja: a modulação não muda a matriz, mas muda quais matrizes pesam no resultado. Por isso a métrica olha só as matrizes do ponto de operação. Ela não precisa de nenhuma simulação nova: usa a referência [1] e as métricas das 140.800 matrizes de cada par, que já estão calculadas.

1. Pela referência [1], achar o ponto de operação de cada configuração e par (aqui, com 1 iteração).
2. Pegar todas as matrizes $\mathbf{G}_c$ dos dois pontos da grade em volta dele. Por exemplo, se a curva cruza em $-9{,}3$ dB, as de -10 e -8 dB. As configurações 5 a 8 usam as matrizes das 1 a 4, porque a modulação não muda $\mathbf{G}_c$.
3. Em cada matriz, anotar o maior valor: a maior parte real ou imaginária, em módulo.
4. Desses valores, pegar o percentil 99, o valor que só 1% das matrizes ultrapassa. O máximo absoluto não é usado porque pode vir de uma única matriz rara, que quase não muda a SER.
5. Os casos com esse número mais alto são os extremos. Se ele passa do limite do formato (± 2048 com $I = 12$, ± 4096 com $I = 13$), o formato satura.

Config.	Par (N, f_c)	Ponto de operação	$\mathbf{G}_c$ (percentil 99)	Perda `<28,12>`	Perda `<25,13>`
8	192; 3,5 GHz	$-8{,}7$ dB	2.470	0,32	1,35
8	132; 3,5 GHz	$-8{,}6$ dB	2.402	0,33	1,48
6	192; 28 GHz	$-9{,}2$ dB	2.387	0,58	1,72
6	168; 28 GHz	$-9{,}1$ dB	2.383	0,58	1,76
6	192; 3,5 GHz	$-9{,}3$ dB	2.266	0,60	1,63
8	24; 3,5 GHz	$-6{,}5$ dB	1.646	0,00	1,06
6	24; 3,5 GHz	$-7{,}2$ dB	1.428	0,00	1,29
10	192; 28 GHz	$-5{,}8$ dB	998	0,00	1,69
12	168; 28 GHz	$-5{,}5$ dB	987	0,01	1,51

Perdas em dB, com AP_TRN e 1 iteração, medidas nos candidatos [5]. `<28,12>` mostra a saturação ($I = 12$); `<25,13>`, a falta de precisão ($F = 12$). A tabela mostra só alguns dos 80 casos: os do topo, os do par com $N = 24$ e os com $M_c = 32$.

O que a métrica acerta.

- **Os piores casos.** Os 8 primeiros dos 80 casos são as configurações 6 e 8 com $N \geq 132$, justamente o pior caso dos candidatos.
- **O limite de I.** Em todos os casos com o número acima de 2.048, $I = 12$ perdeu de 0,28 a 0,60 dB. Abaixo disso, perdeu no máximo 0,01 dB, inclusive com $N = 24$. A métrica diz, antes de rodar o hardware, se um I basta.
- **A ordem da precisão.** Quanto maior $\mathbf{G}_c$, menores os valores da inversa, e mais o truncamento pesa. Ordenando os 80 casos pela métrica e pela perda de `<25,13>`, as duas listas ficam quase na mesma ordem (correlação de postos de Spearman de 0,93). O número de condição no ponto de operação deu só 0,68.

Onde ela falha. As configurações 10 e 12 ($M_c = 32$) têm $\mathbf{G}_c$ com cerca de 1.000, metade das 6 e 8, e mesmo assim perdem quase o mesmo em precisão (1,5 a 1,7 dB). Uma explicação provável, não testada à parte: a matriz 32×32 faz mais contas, e os erros de truncamento se acumulam. Por isso a escolha deve incluir uma configuração com $M_c = 32$, mesmo que ela não esteja no topo da lista.

A escolha. Configurações 6 e 8 (QPSK, IoT de 20 dB), mais uma com $M_c = 32$ (10 ou 12, que dão quase o mesmo resultado); pares $(192; 3{,}5\ \text{GHz})$ e $(168; 28\ \text{GHz})$, os de maior perda. O par com $N = 24$ é o mais fácil nos dois eixos: com $N = 24$, a curva cruza 10^{-2} numa SNR mais alta, onde $\mathbf{G}_c$ é menor. Essa escolha foi conferida com os resultados da [5]; a métrica em si não precisa deles.

O resultado na [4]. A [4] usou essa escolha, com a configuração 1 como caso típico e as duas com $M_c = 32$. Os piores casos da [4] foram as configurações 6 e 8, como a métrica previa, e os menores formatos da grade ($I = 14$, $F = 16$) ficaram iguais ao limite da [5].

Pares (N, f_c)

Cada condição de teste da norma (TS 38.104, Anexo G) define duas coisas para o simulador: o número N de amostras de ruído por PRB, que vem do tempo de coerência e do DM-RS, e a frequência da portadora f_c. Condições com o mesmo par dão o mesmo resultado, então cada par foi simulado uma vez.

A portadora muda o resultado porque, no cenário UMa NLOS do QuaDRiGa, o espalhamento angular depende da frequência (cerca de 86° de espalhamento de chegada a 3,5 GHz e 49° a 28 GHz), e isso muda a correlação entre as antenas. A velocidade só entra pelo tempo de coerência, ou seja, pelo N: o canal é considerado constante dentro do bloco de coerência, sem Doppler.

Par	N	f_c	Condições de teste	Mobilidade típica
1	192	3,5 GHz (FR1)	TDLA30-5, TDLA30-10	baixa
2	192	28 GHz (FR2)	TDLA30-75	baixa
3	168	28 GHz (FR2)	TDLA30-300	média a alta
4	132	3,5 GHz (FR1)	TDLC300-100	média a alta
5	24	3,5 GHz (FR1)	TDLB100-400	média a alta

As condições HST350 e HST500 (trem de alta velocidade) ficaram de fora: o tempo de coerência dura um símbolo OFDM, e a maioria dos símbolos de dados não tem nenhuma amostra de ruído ($N = 0$).